

Design

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	Notation	2
2	Introduction	5

Notation

- **Different notations**

1. **Class diagram:** A class diagram depicts classes, their structure, and the static relationships between them.

Each design pattern includes at least one class diagram. The other notations are used as needed to supplement the discussion. The class and object diagrams are based on **OMT (Object Modeling Technique)**

2. **Object diagram:** An object diagram depicts a particular object structure at run-time.

OMT uses the term "object diagram" to refer to class diagrams. We use "object diagram" exclusively to refer to diagrams of object structures

3. **Interaction diagram:** An interaction diagram shows the flow of requests between objects.

- **Class diagrams:** A class is denoted by a box with the class name in bold type at the top. The key operations of the class appear below the class name. Any instance variables appear below the operations. Type information is optional; we use the C++ convention, which puts the type name before the name of the operation (to signify the return type), instance variable, or actual parameter. Slanted type indicates that the class or operation is abstract.



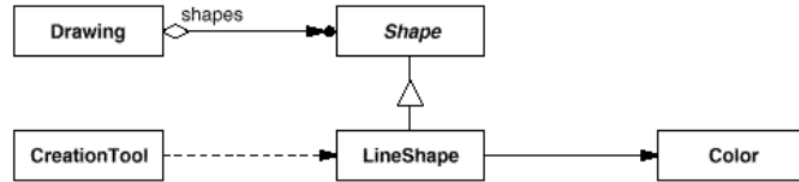
(a) Abstract and concrete classes

In some design patterns it's helpful to see where client classes reference Participant classes. When a pattern includes a Client class as one of its participants (meaning the client has a responsibility in the pattern), the Client appears as an ordinary class. This is true in Flyweight, for example. When the pattern does not include a Client participant (i.e., clients have no responsibilities in the pattern), but including it nevertheless clarifies which pattern participants interact with clients, then the Client class is shown in gray.



(b) Participant Client class (left) and implicit Client class (right)

The OMT notation for class inheritance is a triangle connecting a subclass (LineShape in the figure) to its parent class (Shape). An object reference representing a part-of or aggregation relationship is indicated by an arrow-headed line with a diamond at the base. The arrow points to the class that is aggregated (e.g., Shape). An arrow-headed line without the diamond denotes acquaintance (e.g., a LineShape keeps a reference to a Color object, which other shapes may share). A name for the reference may appear near the base to distinguish it from other references.



(c) Class relationships

Another useful thing to show is which classes instantiate which others. We use a dashed arrow-headed line to indicate this, since OMT doesn't support it. We call this the "**creates**" relationship. The arrow points to the class that's instantiated. CreationTool creates LineShape objects.

OMT also defines a filled circle to mean "more than one." When the circle appears at the head of a reference, it means multiple objects are being referenced or aggregated. Figure B.1c shows that Drawing aggregates multiple objects of type Shape.

Finally, we've augmented OMT with pseudocode annotations to let us sketch the implementations of operations.



(d) Pseudocode annotation

- **Object Diagram:** An object diagram shows instances exclusively. It provides a snapshot of the objects in a design pattern. The objects are named "*aSomething*", where Something is the class of the object. Our symbol for an object (modified slightly from standard OMT) is a rounded box with a line separating the object name from any object references. Arrows indicate the object referenced.

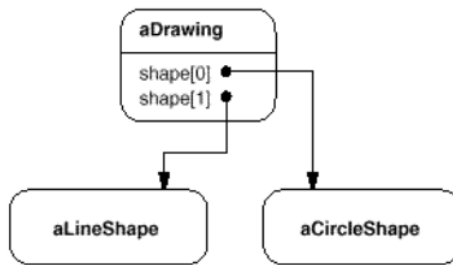
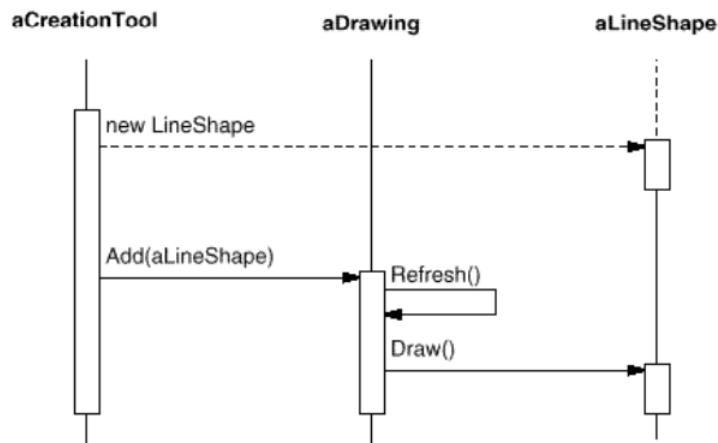


Figure B.2: Object diagram notation

- **Interaction diagrams:** An interaction diagram shows the order in which requests between objects get executed. Below is an interaction diagram that shows how a shape gets added to a drawing.



Time flows from top to bottom in an interaction diagram. A solid vertical line indicates the lifetime of a particular object. The naming convention for objects is the same as for object diagrams; the class name prefixed by the letter "a" (e.g., aShape). If the object doesn't get instantiated until after the beginning of time as recorded in the diagram, then its vertical line appears dashed until the point of creation.

A vertical rectangle shows that an object is active; that is, it is handling a request. The operation can send requests to other objects; these are indicated with a horizontal arrow pointing to the receiving object. The name of the request is shown above the arrow. A request to create an object is shown with a dashed arrow-headed line. A request to the sending object itself points back to the sender.

Introduction

- **Objects:** In Object-Oriented Programming (OOP), an object is represented as a self-contained unit that combines state (data) and behavior (functions). It is a concrete instance of a blueprint known as a class.
 - **State:** Represented by attributes, fields, or properties.
 - **Behavior:** Represented by methods or functions.
 - **Identity:** A unique address or identifier in memory.
- **Abstract vs concrete classes:**
 - **Abstract class:** An abstract class is an incomplete blueprint that cannot be instantiated directly and often contains undefined methods.
 - **Concrete class:** A concrete class is a complete, fully-implemented class that can be instantiated directly to create objects.
- **Participant / Client:** In software architecture, the participant and client classes represent distinct roles. A client acts as the interface initiating an interaction (e.g., an AWS or API connection), while a participant represents an active entity within that session (e.g., a person, agent, or device in a chat or RTC room).
 - **Client:** The client class establishes a connection and handles the communication protocol, authentication, and requests between your application and a backend service.
 - **Participant:** The participant class represents a specific user, agent, or automated bot actively involved in the established session or context.
- **What is a design pattern:** Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice
- **Four essential elements:**
 1. **Pattern name:** The pattern name is a handle we can use to describe a design problem, its solutions, and consequences in a word or two. Naming a pattern immediately increases our design vocabulary. It lets us design at a higher level of abstraction. Having a vocabulary for patterns lets us talk about them with our colleagues, in our documentation, and even to ourselves. It makes it easier to think about designs and to communicate them and their trade-offs to others. Finding good names has been one of the hardest parts of developing our catalog.
 2. **Problem:** The problem describes when to apply the pattern. It explains the problem and its context. It might describe specific design problems such as how to represent algorithms as objects. It might describe class or object structures that are symptomatic of an inflexible design. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
 3. **Solution:** The solution describes the elements that make up the design, their relationships, responsibilities, and collaborations. The solution doesn't describe a particular concrete design or implementation, because a pattern is like a template that can be applied in many different situations. Instead, the pattern provides an abstract description of a design problem and how a general arrangement of elements (classes and objects in our case) solves it.

4. **Consequences:** The consequences are the results and trade-offs of applying the pattern. Though consequences are often unvoiced when we describe design decisions, they are critical for evaluating design alternatives and for understanding the costs and benefits of applying the pattern. The consequences for software often concern space and time trade-offs. They may address language and implementation issues as well. Since reuse is often a factor in object-oriented design, the consequences of a pattern include its impact on a system's flexibility, extensibility, or portability. Listing these consequences explicitly helps you understand and evaluate them.
- **MVC:** MVC consists of three kinds of objects. The Model is the application object, the View is its screen presentation, and the Controller defines the way the user interface reacts to user input.

MVC decouples views and models by establishing a subscribe/notify protocol between them. A view must ensure that its appearance reflects the state of the model. Whenever the model's data changes, the model notifies views that depend on it. In response, each view gets an opportunity to update itself. This approach lets you attach multiple views to a model to provide different presentations. You can also create new views for a model without rewriting it.

This design is applicable to a more general problem: decoupling objects so that changes to one can affect any number of others without requiring the changed object to know details of the others.

MVC also lets you change the way a view responds to user input without changing its visual presentation. You might want to change the way it responds to the keyboard, for example, or have it use a pop-up menu instead of command keys. MVC encapsulates the response mechanism in a Controller object. There is a class hierarchy of controllers, making it easy to create a new controller as a variation on an existing one.

A view uses an instance of a Controller subclass to implement a particular response strategy; to implement a different strategy, simply replace the instance with a different kind of controller. It's even possible to change a view's controller at run-time to let the view change the way it responds to user input. For example, a view can be disabled so that it doesn't accept input simply by giving it a controller that ignores input events.

The main relationships in MVC are given by the Observer, Composite, and Strategy design patterns. MVC uses other design patterns, such as Factory Method (121) to specify the default controller class for a view and Decorator (196) to add scrolling to a view.

- **Describing design patterns:** We describe design patterns using a consistent format. Each pattern is divided into sections according to the following template. The template lends a uniform structure to the information, making design patterns easier to learn, compare, and use.
 1. **Pattern name and classification:** The pattern's name conveys the essence of the pattern succinctly. A good name is vital, because it will become part of your design vocabulary.
 2. **Intent:** A short statement that answers the following questions: What does the design pattern do? What is its rationale and intent? What particular design issue or problem does it address?

3. **Also known as:** Other well-known names for the pattern, if any.
4. **Motivation:** A scenario that illustrates a design problem and how the class and object structures in the pattern solve the problem. The scenario will help you understand the more abstract description of the pattern that follows.
5. **Applicability:** What are the situations in which the design pattern can be applied? What are examples of poor designs that the pattern can address? How can you recognize these situations?
6. **Structure:** A graphical representation of the classes in the pattern using a notation based on the **Object Modeling Technique (OMT)**. We also use **interaction diagrams** to illustrate sequences of requests and collaborations between objects.
7. **Participants:** The classes and/or objects participating in the design pattern and their responsibilities.
8. **Collaborations:** How the participants collaborate to carry out their responsibilities.
9. **Consequences:** How does the pattern support its objectives? What are the trade-offs and results of using the pattern? What aspect of system structure does it let you vary independently?
10. **Implementation:** What pitfalls, hints, or techniques should you be aware of when implementing the pattern? Are there language-specific issues?
11. **Sample code:** Code fragments that illustrate how you might implement the pattern in C++ or Smalltalk.
12. **Known uses:** Examples of the pattern found in real systems.
13. **Related patterns:** What design patterns are closely related to this one? What are the important differences? With which other patterns should this one be used?

- **The catalog of design patterns:**

- **Abstract Factory:** Provide an interface for creating families of related or dependent objects without specifying their concrete classes.
- **Adapter:** Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.
- **Bridge:** Decouple an abstraction from its implementation so that the two can vary independently.
- **Builder:** Separate the construction of a complex object from its representation so that the same construction process can create different representations.
- **Chain of Responsibility:** Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.
- **Command:** Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.
- **Composite:** Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.
- **Decorator:** Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

- **Facade**: Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use.
 - **Factory Method**: Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
 - **Flyweight**: Use sharing to support large numbers of fine-grained objects efficiently.
 - **Interpreter**: Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.
 - **Iterator**: Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
 - **Mediator**: Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.
 - **Memento**: Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later.
 - **Observer**: Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
 - **Prototype**: Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.
 - **Proxy**: Provide a surrogate or placeholder for another object to control access to it.
 - **Singleton**: Ensure a class only has one instance, and provide a global point of access to it.
 - **State**: Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.
 - **Strategy**: Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
 - **Template Method**: Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.
 - **Visitor**: Represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.
- **Organize the catalog**: Design patterns vary in their granularity and level of abstraction. Because there are many design patterns, we need a way to organize them. This section classifies design patterns so that we can refer to families of related patterns. The classification helps you learn the patterns in the catalog faster, and it can direct efforts to find new patterns as well.

We classify design patterns by two criteria

1. **Purpose**: The **first criterion**, called purpose, reflects what a pattern does. Patterns can have either **creational**, **structural**, or **behavioral** purpose. Creational patterns concern the process of object creation. Structural patterns deal with the composition of classes or objects. Behavioral patterns characterize the ways in which classes or objects interact and distribute responsibility.

2. **Scope:** The second criterion, called **scope**, specifies whether the pattern applies primarily to classes or to objects. Class patterns deal with relationships between classes and their subclasses. These relationships are established through inheritance, so they are static—fixed at compile-time. Object patterns deal with object relationships, which can be changed at run-time and are more dynamic. Almost all patterns use inheritance to some extent. So the only patterns labeled "class patterns" are those that focus on class relationships. Note that most patterns are in the Object scope.

Scope	Class	Purpose		
		Creational	Structural	Behavioral
		Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

Table 1.1: Design pattern space

Creational class patterns defer some part of object creation to subclasses, while Creational object patterns defer it to another object.

The Structural class patterns use inheritance to compose classes, while the Structural object patterns describe ways to assemble objects.

The Behavioral class patterns use inheritance to describe algorithms and flow of control, whereas the Behavioral object patterns describe how a group of objects cooperate to perform a task that no single object can carry out alone.

There are other ways to organize the patterns. Some patterns are often used together. For example, Composite is often used with Iterator or Visitor. Some patterns are alternatives: Prototype is often an alternative to Abstract Factory. Some patterns result in similar designs even though the patterns have different intents. For example, the structure diagrams of Composite and Decorator are similar.

Yet another way to organize design patterns is according to how they reference each other in their "Related Patterns" sections.

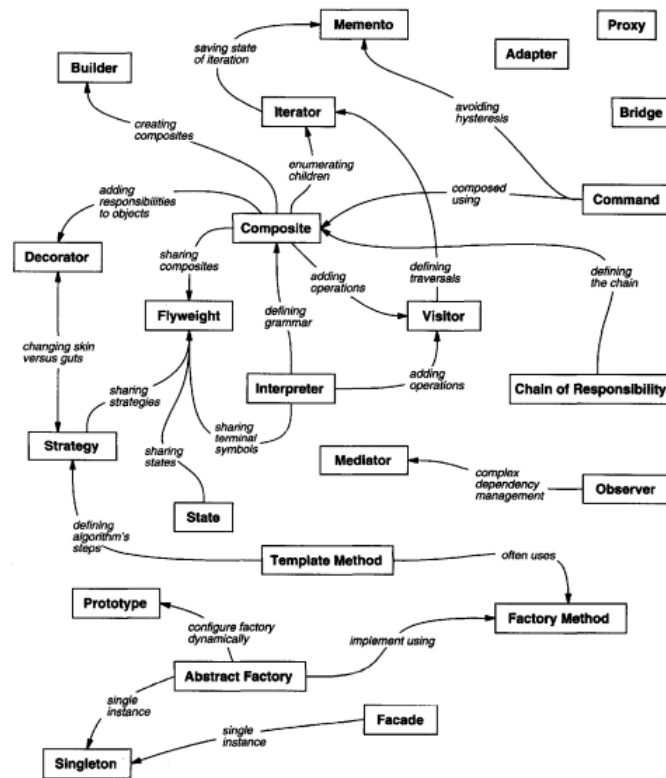


Figure 1.1: Design pattern relationships

- **Finding Appropriate Objects:** Object-oriented programs are made up of objects. An object packages both data and the procedures that operate on that data. The procedures are typically called **methods** or operations. An object performs an operation when it receives a request (or **message**) from a **client**.

Requests are the only way to get an object to execute an operation. Operations are the only way to change an object's internal data. Because of these restrictions, the object's internal state is said to be encapsulated; it cannot be accessed directly, and its representation is invisible from outside the object.

The hard part about object-oriented design is decomposing a system into objects. The task is difficult because many factors come into play: encapsulation, granularity, dependency, flexibility, performance, evolution, reusability, and on and on. They all influence the decomposition, often in conflicting ways.

Object-oriented design methodologies favor many different approaches. You can;

- Write a problem statement, single out the nouns and verbs, and create corresponding classes and operations.
 - Focus on the collaborations and responsibilities in your system.
 - Model the real world and translate the objects found during analysis into design.
- There will always be disagreement on which approach is best.

Many objects in a design come from the analysis model. But object-oriented designs often end up with classes that have no counterparts in the real world. Strict modeling of the real world leads to a system that reflects today's realities but not necessarily tomorrow's. The abstractions that emerge during design are key to making a design flexible.

Design patterns help you identify less-obvious abstractions and the objects that can capture them.

- **Determining Object Granularity:** Objects can vary tremendously in size and number. They can represent everything down to the hardware or all the way up to entire applications. How do we decide what should be an object?
- **Specifying Object Interfaces:** Every operation declared by an object specifies the operation's name, the objects it takes as parameters, and the operation's return value. This is known as the operation's signature. The set of all signatures defined by an object's operations is called the interface to the object. An object's interface characterizes the complete set of requests that can be sent to the object. Any request that matches a signature in the object's interface may be sent to the object.

A type is a name used to denote a particular interface. We speak of an object as having the type "Window" if it accepts all requests for the operations defined in the interface named "Window." An object may have many types, and widely different objects can share a type. Part of an object's interface may be characterized by one type, and other parts by other types. Two objects of the same type need only share parts of their interfaces. Interfaces can contain other interfaces as subsets. We say that a type is a subtype of another if its interface contains the interface of its supertype. Often we speak of a subtype inheriting the interface of its supertype.

Interfaces are fundamental in object-oriented systems. Objects are known only through their interfaces. There is no way to know anything about an object or to ask it to do anything without going through its interface. An object's interface says nothing about its implementation, different objects are free to implement requests differently. That means two objects having completely different implementations can have identical interfaces.

When a request is sent to an object, the particular operation that's performed depends on both the request and the receiving object. Different objects that support identical requests may have different implementations of the operations that fulfill these requests. The run-time association of a request to an object and one of its operations is known as dynamic binding.

Dynamic binding means that issuing a request doesn't commit you to a particular implementation until run-time.

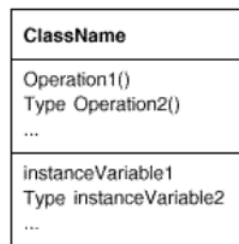
Moreover, dynamic binding lets you substitute objects that have identical interfaces for each other at run-time. This substitutability is known as polymorphism, and it's a key concept in object-oriented systems. It lets a client object make few assumptions about other objects beyond supporting a particular interface. Polymorphism simplifies the definitions of clients, decouples objects from each other, and lets them vary their relationships to each other at run-time.

Design patterns help you define interfaces by identifying their key elements and the kinds of data that get sent across an interface. A design pattern might also tell you what not to put in the interface.

Design patterns also specify relationships between interfaces. In particular, they often require some classes to have similar interfaces, or they place constraints on the interfaces of some classes.

- **Specifying Object Implementations:** So far we've said little about how we actually define an object. An object's implementation is defined by its class. The class specifies the object's internal data and representation and defines the operations the object can perform.

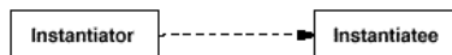
Our OMT-based notation depicts a class as a rectangle with the class name in bold. Operations appear in normal type below the class name. Any data that the class defines comes after the operations. Lines separate the class name from the operations and the operations from the data.



Return types and instance variable types are optional, since we don't assume a statically typed implementation language.

Objects are created by **instantiating** a class. The object is said to be an **instance** of the class. The process of instantiating a class allocates storage for the object's internal data (made up of instance variables) and associates the operations with these data. Many similar instances of an object can be created by instantiating a class.

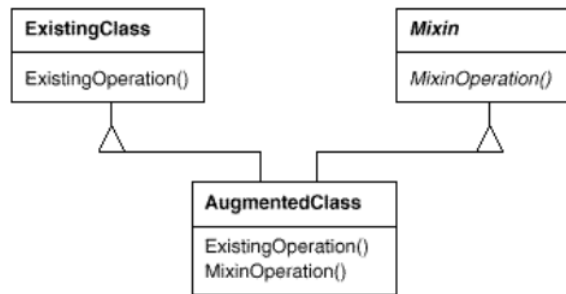
A dashed arrowhead line indicates a class that instantiates objects of another class. The arrow points to the class of the instantiated objects



An abstract class is one whose main purpose is to define a common interface for its subclasses. An abstract class will defer some or all of its implementation to operations defined in subclasses; hence an abstract class cannot be instantiated.

The operations that an abstract class declares but doesn't implement are called abstract operations. Classes that aren't abstract are called concrete classes.

A mixin class is a class that's intended to provide an optional interface or functionality to other classes. It's similar to an abstract class in that it's not intended to be instantiated. Mixin classes require multiple inheritance:



- **Class versus interface inheritance:** It's important to understand the difference between an object's class and its type. An object's class defines how the object is implemented. The class defines the object's internal state and the implementation of its operations. In contrast, an object's type only refers to its interface—the set of requests to which it can respond. An object can have many types, and objects of different classes can have the same type.

Of course, there's a close relationship between class and type. Because a class defines the operations an object can perform, it also defines the object's type. When we say that an object is an instance of a class, we imply that the object supports the interface defined by the class.

It's also important to understand the difference between class inheritance and interface inheritance (or subtyping). Class inheritance defines an object's implementation in terms of another object's implementation. In short, it's a mechanism for code and representation sharing. In contrast, interface inheritance (or subtyping) describes when an object can be used in place of another.

It's easy to confuse these two concepts, because many languages don't make the distinction explicit. In languages like C++ and Eiffel, inheritance means both interface and implementation inheritance. The standard way to inherit an interface in C++ is to inherit publicly from a class that has (pure) virtual member functions. Pure interface inheritance can be approximated in C++ by inheriting publicly from pure abstract classes. Pure implementation or class inheritance can be approximated with private inheritance.

Although most programming languages don't support the distinction between interface and implementation inheritance, people make the distinction in practice.

- **Programming to an Interface, not an Implementation:** Class inheritance is basically just a mechanism for extending an application's functionality by reusing functionality in parent classes. It lets you define a new kind of object rapidly in terms of an old one. It lets you get new implementations almost for free, inheriting most of what you need from existing classes.

However, implementation reuse is only half the story. Inheritance's ability to define families of objects with identical interfaces (usually by inheriting from an abstract class) is also important. Why? Because polymorphism depends on it.